

Mathematical Modeling of Bitcoin Private-Chain Attacks

Private double spending, and selfish mining

Siyu Liu

2026/6/16

Notation and core assumptions

Hashrate split

q = attacker hashrate fraction, $p = 1 - q$ = honest hashrate fraction.

Block arrivals

Let the whole network produce blocks at rate λ . By Poisson thinning,

$$H(t) \sim \text{Pois}(\lambda p t), \quad A(t) \sim \text{Pois}(\lambda q t),$$

and $H(t)$ and $A(t)$ are independent.

Recall the Poisson probability mass function is

$$\mathbb{P}[X = k] = e^{-\theta} \frac{\theta^k}{k!}, \quad k = 0, 1, 2, \dots,$$

for $X \sim \text{Pois}(\theta)$.

Attack workflow

1. The attacker broadcasts a payment transaction to the merchant.
2. At the same time as the transaction is included on the public chain, the attacker secretly mines a private fork with a conflicting transaction.
3. The merchant waits for z public confirmations and then accepts the payment.
4. If the private fork catches up with or overtakes the public chain, the attacker publishes it.
5. The network follows the longer chain, so the merchant's payment can be reversed.

Stale-chain catch-up

Suppose a private branch is $d \geq 1$ blocks behind. Let

$$\psi_d = \mathbb{P}(\text{the attacker ever catches up} \mid \text{attacker left behind } d \text{ blocks at first}).$$

Consider whether the attacker or the honest participant produces the first block.

$$\psi_d = q\psi_{d-1} + p\psi_{d+1}, \quad \psi_0 = 1, \quad \lim_{d \rightarrow \infty} \psi_d = 0.$$

Solving the linear recurrence gives

$$\psi_d = \begin{cases} (q/p)^d, & q < p, \\ 1, & q \geq p. \end{cases}$$

This is the basic *stale-chain catch-up* probability.

Private double-spend attack

The attacker pays a merchant on the public chain while secretly mining a conflicting private chain.

1. The merchant waits for z honest confirmations.
2. During that waiting time, the attacker mines K private blocks.
3. If $K \geq z$, the attacker can publish a private chain at least as long as the public chain.
4. If $K < z$, the attacker is $z - K$ blocks behind and succeeds with probability $(q/p)^{z-K}$.

Thus

$$P_{\text{priv}}(z, q) = \mathbb{E} \left[\mathbf{1}_{K \geq z} + \mathbf{1}_{K < z} \left(\frac{q}{p} \right)^{z-K} \right].$$

Distribution of K : exact negative-binomial law

Let T_z be the time when the honest chain finds its z -th block. Since honest blocks arrive at rate λp ,

$$T_z \sim \text{Gamma}(z, \lambda p).$$

Its density function is

$$f_{T_z}(t) = \frac{(\lambda p)^z}{(z-1)!} t^{z-1} e^{-\lambda p t}, \quad t > 0.$$

Conditioned on $T_z = t$, the attacker mines

$$K \mid T_z = t \sim \text{Pois}(\lambda q t).$$

Mixing Poisson with Gamma gives

$$\mathbb{P}[K = k] = \int_0^\infty e^{-\lambda q t} \frac{(\lambda q t)^k}{k!} \frac{(\lambda p)^z t^{z-1} e^{-\lambda p t}}{(z-1)!} dt.$$

Therefore

$$\mathbb{P}[K = k] = \binom{z+k-1}{k} p^z q^k, \quad k = 0, 1, 2, \dots$$

Private-attack success probability

Plug the negative-binomial law into the conditional catch-up probability:

$$P_{\text{priv}}(z, q) = \sum_{k=0}^{z-1} \binom{z+k-1}{k} p^z q^k \left(\frac{q}{p}\right)^{z-k} + \sum_{k=z}^{\infty} \binom{z+k-1}{k} p^z q^k.$$

Equivalently,

$$P_{\text{priv}}(z, q) = 1 - \sum_{k=0}^{z-1} \binom{z+k-1}{k} (p^z q^k - p^k q^z).$$

A compact special-function form is

$$P_{\text{priv}}(z, q) = I_{pq}\left(z, \frac{1}{2}\right), \quad q < p,$$

where $I_x(a, b)$ is the regularized incomplete beta function, with

$$I_x(a, b) = \frac{B_x(a, b)}{B(a, b)} = \frac{\int_0^x t^{a-1} (1-t)^{b-1} dt}{\int_0^1 t^{a-1} (1-t)^{b-1} dt}.$$

Comparison with Nakamoto's Poisson approximation

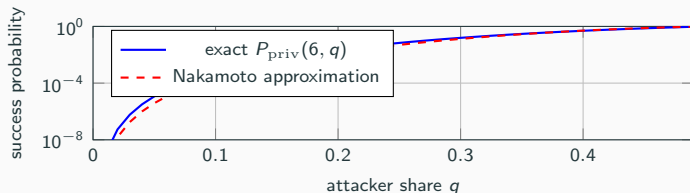
Nakamoto approximates K by

$$K \approx \text{Pois}(\mu), \quad \mu = zq/p.$$

Then

$$P_{\text{Nak}}(z, q) = 1 - \sum_{k=0}^z e^{-\mu} \frac{\mu^k}{k!} \left(1 - \left(\frac{q}{p} \right)^{z-k} \right).$$

For $z = 6$, using the exact negative-binomial value is more conservative from a defender's viewpoint.



Exponential decay in confirmations

For fixed $q < p$, the exact probability has the asymptotic form

$$P_{\text{priv}}(z, q) = I_{4pq}\left(z, \frac{1}{2}\right) \sim \frac{(4pq)^z}{(p-q)\sqrt{\pi z}} \quad (z \rightarrow \infty).$$

Therefore confirmations buy security exponentially fast in z , but the exponent deteriorates as q approaches $1/2$.

Given a tolerated attack probability ε , define

$$q^*(z, \varepsilon) = \sup\{q < 1/2 : P_{\text{priv}}(z, q) \leq \varepsilon\}.$$

z confirmations	ε	max q
3	10^{-3}	3.76%
6	10^{-3}	11.00%
10	10^{-3}	17.38%
20	10^{-3}	25.52%
30	10^{-3}	29.61%

Selfish Mining

Hashrate split

$\alpha = \text{attacker/selfish hashrate}, \quad h = 1 - \alpha = \text{honest hashrate}.$

Tie-breaking parameter

When two public branches have the same height,

$\gamma = \text{fraction of honest miners mining on the attacker's branch}.$

Private lead

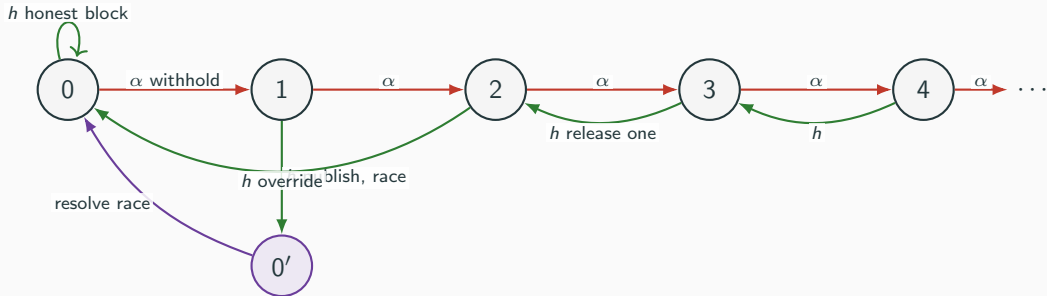
$\ell = \text{length of attacker's private chain} - \text{length of public honest chain}.$

States are

$\ell = 0, 1, 2, 3, \dots$ plus a special tie state $0'$.

Maintain a pair (S_ℓ, L_ℓ) : (selfish, accepted blocks, total accepted blocks)

State machine overview



- Red transitions: attacker mines the next block and usually withholds it.
- Green transitions: honest miners find the next block; the attacker reacts.
- Purple state: a public race, where γ matters.

State 0: no private advantage

Honest block, probability h

The honest block is immediately accepted.

$0 \rightarrow 0$, reward $(0, 1)$.

Attacker block, probability α

The attacker withholds the block and starts a private fork.

$0 \rightarrow 1$, reward $(0, 0)$.

No block reward is settled yet.

$$(S_0, L_0) = h(0, 1) + \alpha(S_1, L_1).$$

State 1: one hidden block

Attacker mines, probability α
The attacker extends the private chain.

$$1 \rightarrow 2, \quad \text{reward } (0, 0).$$

The attack now has a safer two-block lead.

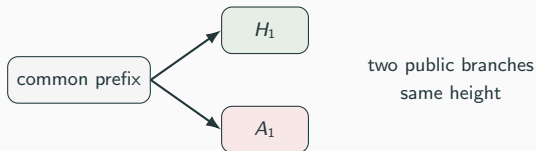
Honest mines, probability h
The honest chain catches up. The attacker publishes the hidden block to create a race.

$$1 \rightarrow 0'.$$

This is the only genuinely dangerous public tie.

$$(S_1, L_1) = \alpha(S_2, L_2) + h(S_{0'}, L_{0'}).$$

Tie state $0'$: where γ matters



From $0'$, the next block resolves the race:

$$(S_{0'}, L_{0'}) = \alpha(2, 2) + h\gamma(1, 2) + h(1 - \gamma)(0, 2).$$

Event	Probability	Accepted main-chain blocks
attacker mines next	α	$A_1 + A_2$, selfish gets 2
honest mines on attacker branch	$h\gamma$	$A_1 + H_2$, selfish gets 1
honest mines on honest branch	$h(1 - \gamma)$	$H_1 + H_2$, selfish gets 0

$$(S_{0'}, L_{0'}) = (2\alpha + h\gamma, 2).$$

State 2: override

Attacker mines, probability α
The attacker keeps withholding.

$2 \rightarrow 3,$ reward $(0, 0)$.

Honest mines, probability h
The attacker publishes the two-block private chain and overrides the honest block.

$2 \rightarrow 0,$ reward $(2, 2)$.

The honest block becomes stale.

$$(S_2, L_2) = \alpha(S_3, L_3) + h(2, 2).$$

Why publish both blocks at lead 2?

If the attacker published only one block, the public view would be a real tie with only one hidden backup block left. Publishing both makes the attacker's chain strictly longer immediately.

State $\ell \geq 3$: controlled release of inventory

At large lead, an honest block does not endanger the attacker.

Attacker mines, probability α
The private lead grows.

$$\ell \rightarrow \ell + 1, \quad \text{reward } (0, 0).$$

Honest mines, probability h
The attacker releases one private block.

$$\ell \rightarrow \ell - 1, \quad \text{reward } (1, 1).$$

The released block is eventually secured, while
the attacker still keeps a private tail.

$$(S_\ell, L_\ell) = \alpha(S_{\ell+1}, L_{\ell+1}) + h((1, 1) + (S_{\ell-1}, L_{\ell-1})), \quad \ell \geq 3.$$

Important subtlety: a surface tie is not state 0'

Suppose $\ell = 3$. The attacker has hidden blocks

$$A_1, A_2, A_3.$$

An honest miner finds H_1 . The attacker publishes only A_1 :

$$P - H_1 \quad \text{versus} \quad P - A_1.$$

This looks like a tie publicly, but the attacker still has A_2, A_3 hidden.

Why no γ here?

Even if honest miners temporarily mine on H_1 , the attacker can reveal more private blocks and maintain a strictly longer chain. Hence A_1 is eventually accepted with probability 1.

state 0' is only the lead-1 race, where the attacker has no backup block.

Selfish mining strategy

1. If the attacker finds a block, withhold it and increase the private lead.
2. If honest miners find a block:
 - $\ell = 0$: accept the honest block.
 - $\ell = 1$: publish the hidden block and enter the public race state $0'$.
 - $\ell = 2$: publish the two-block private chain and override the honest block.
 - $\ell \geq 3$: publish one private block, collect one secured reward, and reduce lead by one.
3. In state $0'$, the next block resolves the race; γ determines how much honest hashrate mines on the attacker's branch.

From state machine to Markov reward equations

Let

$$S_\ell = \mathbb{E}[\text{selfish accepted blocks before returning to 0} \mid \ell],$$

$$L_\ell = \mathbb{E}[\text{total accepted blocks before returning to 0} \mid \ell].$$

The state machine gives

$$(S_0, L_0) = h(0, 1) + \alpha(S_1, L_1),$$

$$(S_1, L_1) = \alpha(S_2, L_2) + h(S_{0'}, L_{0'}),$$

$$(S_{0'}, L_{0'}) = \alpha(2, 2) + h\gamma(1, 2) + h(1 - \gamma)(0, 2),$$

$$(S_2, L_2) = \alpha(S_3, L_3) + h(2, 2),$$

$$(S_\ell, L_\ell) = \alpha(S_{\ell+1}, L_{\ell+1}) + h((1, 1) + (S_{\ell-1}, L_{\ell-1})), \quad \ell \geq 3.$$

$$\text{Long-run selfish revenue ratio} = \frac{S_0}{L_0}.$$

Selfish mining revenue and threshold

Solving the recurrences gives the Eyal-Sirer revenue formula

$$R(\alpha, \gamma) = \frac{\alpha(1-\alpha)^2(4\alpha + \gamma(1-2\alpha)) - \alpha^3}{1 - \alpha[1 + \alpha(2 - \alpha)]}.$$

Selfish mining is profitable when $R(\alpha, \gamma) > \alpha$. Algebra gives

$$R(\alpha, \gamma) - \alpha = \frac{\alpha(1-\alpha)^2((3-2\gamma)\alpha + \gamma - 1)}{1 - \alpha - 2\alpha^2 + \alpha^3}.$$

Thus

$$\alpha > \alpha^*(\gamma) = \frac{1-\gamma}{3-2\gamma}.$$

Examples:

$$\gamma = 0 : \alpha^* = 1/3, \quad \gamma = 1/2 : \alpha^* = 1/4, \quad \gamma = 1 : \alpha^* = 0.$$

1. Block production is naturally modeled by two independent Poisson processes; after embedding at block times, it becomes a biased random walk.
2. A stale chain d blocks behind catches up with probability $(q/p)^d$ when $q < p$.
3. The exact private double-spend success probability after z confirmations is

$$P_{\text{priv}}(z, q) = I_{4pq}\left(z, \frac{1}{2}\right).$$

4. Selfish mining has a different risk profile: it becomes profitable above

$$\alpha^*(\gamma) = \frac{1 - \gamma}{3 - 2\gamma}.$$

Three-Party Selfish Mining

If and only if a participant holds more than $\frac{1}{3}$ of the total hash power when he has the motivation to do selfish mining. Moreover, from a game theory prospective, he will do so as well.

However, when more than one party play selfish mining, the situation becomes more complicated. We consider this situation where each participant holding more than $\frac{1}{3}$ of the total hash power plays selfish mining strategy as aforementioned, regarding all the other miners honest. So there are at most two participants play selfish mining. Together with the honest party, we call this "Three-Party Selfish Mining".

There are three mining populations:

$$A, \quad B, \quad H.$$

Their relative hashrates are

$$\Pr[A \text{ finds next block}] = \alpha, \quad \Pr[B \text{ finds next block}] = \delta,$$

$$\Pr[H \text{ finds next block}] = h = 1 - \alpha - \delta.$$

State variables

A state is

$$x = (\ell_A, \ell_B, \mathcal{F}).$$

- ℓ_A : number of hidden private blocks of A above the current public trunk.
- ℓ_B : number of hidden private blocks of B above the current public trunk.
- $\mathcal{F}$: visible public fork set. If no public race is active, $\mathcal{F} = \emptyset$.

The regenerative base state is

$$x_0 = (0, 0, \emptyset).$$

Rewards are written as

(accepted blocks of A , accepted blocks of B , total accepted blocks).

Why we need public fork states

Single-attacker selfish mining has one special tie state:

A -branch vs. H -branch.

With two independent attackers, we can see several visible races:

A -branch vs. H -branch,

B -branch vs. H -branch,

A -branch vs. B -branch,

A -branch vs. B -branch vs. H -branch.

So the scalar parameter γ becomes a branch-choice rule over all visible branches.

Branch-choice rule

If a public fork has visible branches

$$\mathcal{F} = \{b_1, \dots, b_r\},$$

then for every mining population $M \in \{A, B, H\}$, define

$$\Gamma_M(b \mid \mathcal{F}) = \Pr[M \text{ mines on branch } b \mid \mathcal{F}].$$

for notation simplicity.

In fact, for $M \in \{A, B\}$, if $M \in \mathcal{F}$

$$\Gamma_M(M \mid \mathcal{F}) = 1$$

The one-attacker parameter is the special case

$$\Gamma(A\text{-branch} \mid \{A, H\}) = \gamma, \quad \Gamma(H\text{-branch} \mid \{A, H\}) = 1 - \gamma.$$

State machine: no public race

In a hidden-state $(\ell_A, \ell_B, \emptyset)$:

- If A finds the next block:

$$(\ell_A, \ell_B, \emptyset) \rightarrow (\ell_A + 1, \ell_B, \emptyset).$$

- If B finds the next block:

$$(\ell_A, \ell_B, \emptyset) \rightarrow (\ell_A, \ell_B + 1, \emptyset).$$

- If H finds the next block: both attackers react using the publication rule. The resulting visible branches are compared by length.

If one published branch is strictly longest, it becomes the new public chain; all incompatible hidden branches are discarded. If several published branches tie for longest, the system enters a public race state $\mathcal{F} \neq \emptyset$.

Public race transition rule

For a public race $\mathcal{F}$, suppose the next block is found by

$$M \in \{A, B, H\}$$

and it extends branch $b \in \mathcal{F}$.

Define reward basis vectors

$$\mathbf{e}_A = (1, 0, 1), \quad \mathbf{e}_B = (0, 1, 1), \quad \mathbf{e}_H = (0, 0, 1).$$

The accepted reward vector is

$$\mathbf{e}_b + \mathbf{e}_M,$$

The probability of this event is

$$\Pr[M] \cdot \Gamma_M(b \mid \mathcal{F}).$$

Bellman equations

For every state x , let

$$r_A(x, y), \quad r_B(x, y), \quad r_T(x, y)$$

be the immediate accepted rewards in a transition $x \rightarrow y$.

The Bellman reward equations are

$$S_A(x) = \sum_y P(x, y) (r_A(x, y) + \mathbf{1}_{y \neq x_0} S_A(y)),$$

$$S_B(x) = \sum_y P(x, y) (r_B(x, y) + \mathbf{1}_{y \neq x_0} S_B(y)),$$

$$L(x) = \sum_y P(x, y) (r_T(x, y) + \mathbf{1}_{y \neq x_0} L(y)).$$

The indicator stops the renewal cycle at the first return to x_0 .

Matrix solution

List the states in a finite truncation $\mathcal{X}_K$, including x_0 . Define

$$Q_{xy} = P(x, y)\mathbf{1}_{y \neq x_0}, \quad x, y \in \mathcal{X}_K.$$

Define expected one-step reward vectors

$$r_A(x) = \sum_y P(x, y)r_A(x, y),$$

$$r_B(x) = \sum_y P(x, y)r_B(x, y), \quad r_T(x) = \sum_y P(x, y)r_T(x, y).$$

Then

$$\mathbf{S}_A = (I - Q)^{-1}\mathbf{r}_A, \quad \mathbf{S}_B = (I - Q)^{-1}\mathbf{r}_B, \quad \mathbf{L} = (I - Q)^{-1}\mathbf{r}_T.$$

Revenue shares

The long-run revenue shares are the renewal-reward ratios

$$\rho_A = \frac{S_A(x_0)}{L(x_0)}, \quad \rho_B = \frac{S_B(x_0)}{L(x_0)}.$$

The honest miners receive

$$\rho_H = 1 - \rho_A - \rho_B.$$

Selfish mining is individually profitable for A if

$$\rho_A > \alpha.$$

It is profitable for B if

$$\rho_B > \delta.$$

Profitability Threshold for Symmetric Selfish Miners

Assume two attackers are symmetric and all branch choices are uniformly random.

The profitability threshold per attacker is approximately

$$a^* \approx 21\% - 22\%.$$

For single selfish miner setting with same assumption, his hash power has to account for 25% of the total hash power.

References



Satoshi Nakamoto.

Bitcoin: A Peer-to-Peer Electronic Cash System.

2008.



Ittay Eyal and Emin G. Sirer.

Majority is Not Enough: Bitcoin Mining is Vulnerable.

Financial Cryptography and Data Security, 2014. arXiv:1311.0243.



Cyril Grunspan and Ricardo Perez-Marco.

Double Spend Races.

International Journal of Theoretical and Applied Finance, 2018. arXiv:1702.02867.



Cyril Grunspan and Ricardo Perez-Marco.

Satoshi Risk Tables.

arXiv:1702.04421, 2017.



Q. Bai, Y. Xu, N. Liu, and X. Wang,

Blockchain Mining with Multiple Selfish Miners,

arXiv:2112.10454v3, 2023